

Deep Learning for NLP

Student name: *sdi2200160*

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Contents

1	Abstract	2
2	Task and Data	2
2.1	Dataset and Labels	2
2.2	From Homework 1 to Homework 2	3
2.3	Preprocessing and Input Construction	3
2.4	Compact Exploratory Analysis	3
3	Method	4
3.1	Shared Pipeline Structure	4
3.2	Transformer Components	4
3.3	Training Loop	4
4	Experimental Protocol	5
4.1	Compared Models	5
4.2	Validation Strategy	5
4.3	Default Training Recipe	5
4.4	Tiny Uniform Sanity Sweep	6
5	Evaluation Protocol	6
6	Results	7
6.1	Homework 1 Reference Point	7
6.2	Transformer Test Results	7
6.3	Interpretation Template	8
7	Subgroup Analysis	8
8	Error Analysis	9
8.1	Expected Error Patterns	9
8.2	Representative Mistakes	9
9	Discussion	9
9.1	Small Non-Improving Changes	10
10	Conclusion	10

1. Abstract

We study **response clarity classification** on the QEvasion political interview dataset [1]. Each example is a question–answer pair, and the target label is one of *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*. Homework 1 approached this task with non-contextual representations such as TF–IDF and averaged GloVe embeddings combined with Logistic Regression. Homework 2 keeps the same high-level pipeline abstraction, but replaces the source encoder with a pretrained transformer tokenizer and replaces the linear model with a fine-tuned transformer sequence classifier.

The goal of this report is not to present an aggressively optimized system. Instead, we document a **simple and principled transformer baseline**: raw question–answer pairs, sentence-pair tokenization, one fixed stratified validation split, AdamW optimization, class-weighted loss, gradient clipping, and checkpoint selection by validation macro F1. We compare the three required pretrained checkpoints—BERT-base [2], DistilBERT [5], and DeBERTa-v3-base [3]. The methodology is finalized below, while the run-specific values are left as explicit placeholders to be filled after the GPU notebooks finish executing.

2. Task and Data

2.1. Dataset and Labels

We use the QEvasion dataset from Hugging Face [1]. Each supervised example contains:

- a segmented journalist question,
- the corresponding interview_answer,
- the target label clarity_label.

The task is three-way classification:

- **Clear Reply**,
- **Ambivalent**,
- **Clear Non-Reply**.

As in Homework 1, we use the official train/test split provided by the dataset. The training split is used for fitting and internal validation, while the official test split is used only for final evaluation and Kaggle submission export.

Split	Instances	Usage
Train	[fill: official train count]	fitting + internal validation
Test	[fill: official test count]	final reporting only

Table 1: Dataset split summary. Fill the exact instance counts from the notebook output.

2.2. From Homework 1 to Homework 2

The conceptual pipeline remains the same across the two assignments:

1. preprocess the raw input,
2. encode the source text,
3. encode the target labels,
4. fit a classifier,
5. decode predictions and evaluate.

This continuity is deliberate. Homework 2 should read as a clean extension of Homework 1 rather than a completely different implementation. The important change is therefore the *representation*: Homework 1 used concatenated text plus non-contextual encoders, while Homework 2 keeps the question and answer as a pair and lets the transformer model learn contextual interactions between them.

2.3. Preprocessing and Input Construction

Preprocessing is intentionally lighter than in Homework 1. We do *not* apply lemmatization, stopwords removal, or handcrafted punctuation normalization. Instead, we keep the raw question and answer text, replace missing values with empty strings, and strip only outer whitespace.

This design is natural for transformer fine-tuning because the pretrained checkpoints were trained on raw text and expect model-specific special tokens and subword segmentation. Each example is passed to the tokenizer as a sentence pair:

(question, answer)

which is internally converted to the checkpoint-specific format. For BERT-like models, this is conceptually similar to:

[CLS] question [SEP] answer [SEP].

Relative to Homework 1, this is a principled simplification: instead of manually engineering lexical features, we rely on the pretrained tokenizer and contextual encoder to model the interaction between the question and the response.

2.4. Compact Exploratory Analysis

The notebook keeps the exploratory analysis compact and focused on factors that matter for model behavior:

- class imbalance,
- question length distribution,
- answer length distribution,
- combined text-length variation across splits.

As in Homework 1, class imbalance remains important. *Clear Reply* is the majority class, while *Ambivalent* is the smallest and typically hardest class. This motivates both macro-averaged evaluation and class-weighted training.

3. Method

3.1. Shared Pipeline Structure

We reuse the same thin Classifier wrapper idea from Homework 1. The wrapper coordinates:

- preprocessing,
- source encoding,
- target encoding,
- model fitting,
- prediction and scoring.

This matters pedagogically. The new report can then argue that the conceptual pipeline is stable while only the encoder and model components change.

3.2. Transformer Components

Homework 2 introduces two new central components:

- `TokenizerEncoder`, which wraps a Hugging Face `AutoTokenizer` and converts each (question, answer) pair into transformer inputs;
- `TransformerModel`, which wraps the Hugging Face sequence-classification auto-model and fine-tunes it using an explicit PyTorch loop.

We do *not* use the Hugging Face Trainer API. The training loop is written manually, which keeps the mechanics visible and stays aligned with the assignment's spirit.

3.3. Training Loop

The training loop is intentionally minimal:

- optimizer: AdamW [4],
- loss: weighted cross-entropy,
- model selection: best validation macro F1,
- regularization: weight decay and gradient clipping,
- device: CUDA only.

Class weights are computed from the training labels using the standard balanced-class formula:

$$w_c = \frac{N}{K \cdot n_c},$$

where N is the number of training examples, K is the number of classes, and n_c is the number of training examples belonging to class c .

After each epoch, we evaluate on the fixed validation split and keep the best checkpoint by validation macro F1. This means the reported final model is not merely the last epoch, but the best epoch observed during training.

4. Experimental Protocol

4.1. Compared Models

We compare the three required pretrained checkpoints:

- bert-base-uncased [2],
- distilbert-base-uncased [5],
- microsoft/deberta-v3-base [3].

All three are fine-tuned as three-class sequence classifiers under the same overall recipe.

4.2. Validation Strategy

Homework 1 used 3-fold cross-validation because the classical models were relatively cheap to train. Homework 2 uses transformer fine-tuning, so repeating full training several times per hyperparameter setting would make the workflow much heavier. We therefore replace cross-validation with a single **15% stratified validation split** taken from the official training split.

This yields a simple and defensible procedure:

- 85% of the official training split is used for gradient updates,
- 15% is used for validation,
- the official test split is never used for model selection.

4.3. Default Training Recipe

The shared default recipe is:

- learning rate: 2×10^{-5} ,
- batch size: 16,
- weight decay: 0.01,
- maximum gradient norm: 1.0,
- validation fraction: 0.15,
- random seed: 42,
- class weighting: enabled.

Parameter	Value	Role
Learning rate	2×10^{-5}	conservative fine-tuning step size
Batch size	16	fits standard Kaggle GPU memory budgets
Weight decay	0.01	mild regularization
Max gradient norm	1.0	stabilizes training
Validation split	15% stratified	checkpoint selection
Class weights	enabled	partially offsets imbalance
Random seed	42	reproducibility

Table 2: Shared default training recipe.

4.4. Tiny Uniform Sanity Sweep

The only tuning we allow is the same small sweep for each required model:

- epochs $\in \{3, 4\}$,
- maximum sequence length $\in \{128, 256, 384\}$.

This produces 6 runs per model, or 18 runs in total across BERT, DistilBERT, and DeBERTa. The sweep is intentionally small. Its role is not exhaustive optimization, but a sanity check over two fundamental choices:

- **training duration**, via the number of epochs;
- **visible context**, via the maximum input length.

Because the same 2×3 search space is used for all three models, the tuning budget remains uniform across architectures.

Model	Best Epochs	Best Max Length	Best Validation Macro F1	Notes
BERT	[fill: 3 or 4]	[fill: 128 / 256 / 384]	[fill: value]	[fill: brief note]
DistilBERT	[fill: 3 or 4]	[fill: 128 / 256 / 384]	[fill: value]	[fill: brief note]
DeBERTa-v3	[fill: 3 or 4]	[fill: 128 / 256 / 384]	[fill: value]	[fill: brief note]

Table 3: Best configuration selected by the tiny sweep for each model. Fill from the notebook sweep summary.

5. Evaluation Protocol

We report the same metrics as in Homework 1:

- Accuracy,
- Macro Precision,
- Macro Recall,

- Macro F1.

Macro F1 remains the primary metric because it treats all classes equally and is therefore more informative than plain accuracy on an imbalanced three-class task.

In addition to overall scores, the notebook also produces:

- confusion matrices,
- learning curves for train loss, validation loss, and validation macro F1,
- subgroup results by question length and answer length,
- representative misclassifications for qualitative error analysis.

6. Results

6.1. Homework 1 Reference Point

We do not rerun the classical baselines inside Homework 2. Instead, we compare against the main Homework 1 reference values already reported there:

Homework 1 Model	Macro F1
TF-IDF + Logistic Regression	0.45
GloVe Wiki + Logistic Regression	0.39
GloVe Twitter + Logistic Regression	0.38

Table 4: Homework 1 reference results used for cross-homework comparison.

These values give the baseline that Homework 2 is trying to improve upon. The strongest classical point of comparison is TF-IDF + Logistic Regression with macro F1 0.45.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
BERT	[fill: value]	[fill: value]	[fill: value]	[fill: value]
DistilBERT	[fill: value]	[fill: value]	[fill: value]	[fill: value]
DeBERTa-v3	[fill: value]	[fill: value]	[fill: value]	[fill: value]

Table 5: Final transformer results on the official test split.

Best HW2 Model	Macro F1	Δ vs HW1 TF-IDF	Δ vs best HW1 GloVe
[fill: model name]	[fill: value]	[fill: difference]	[fill: difference]

Table 6: Headline comparison between the strongest Homework 2 model and the main Homework 1 baselines.

6.2. Transformer Test Results

6.3. Interpretation Template

After the notebooks finish, this subsection should be updated with a short paragraph answering the following:

- Which transformer performed best overall?
- Did transformer fine-tuning clearly outperform the Homework 1 GloVe baselines?
- Did it also surpass the stronger TF-IDF baseline?
- Was the gain large enough to count as a meaningful conceptual improvement rather than noise?

A concise fill-in template is:

The best Homework 2 model was *[fill: best model]*, which achieved macro F1 *[fill: value]*. This is *[fill: higher/lower]* than the Homework 1 TF-IDF baseline of 0.45 by *[fill: difference]*, and *[fill: higher/lower]* than the best Homework 1 GloVe baseline by *[fill: difference]*. This suggests that contextual sentence-pair encoding *[fill: did/did not]* provide a clear improvement on this task.

7. Subgroup Analysis

The assignment explicitly asks for performance across data subgroups. We therefore analyze the final predictions across short, medium, and long buckets for both question length and answer length.

Question-Length Bucket	Count	Accuracy	Macro F1
Short	<i>[fill: count]</i>	<i>[fill: value]</i>	<i>[fill: value]</i>
Medium	<i>[fill: count]</i>	<i>[fill: value]</i>	<i>[fill: value]</i>
Long	<i>[fill: count]</i>	<i>[fill: value]</i>	<i>[fill: value]</i>

Table 7: Performance by question-length bucket.

Answer-Length Bucket	Count	Accuracy	Macro F1
Short	<i>[fill: count]</i>	<i>[fill: value]</i>	<i>[fill: value]</i>
Medium	<i>[fill: count]</i>	<i>[fill: value]</i>	<i>[fill: value]</i>
Long	<i>[fill: count]</i>	<i>[fill: value]</i>	<i>[fill: value]</i>

Table 8: Performance by answer-length bucket.

After execution, the report should summarize whether performance degrades more on long answers, short answers, or neither. A useful one-sentence conclusion format is:

The model performs worst on *[fill: short / medium / long]* *[fill: questions / answers]*, suggesting that *[fill: brief explanation from notebook observations]*.

8. Error Analysis

The notebook extracts representative mistakes and the most common confusion pairs. This section should keep the final report qualitative but concrete.

8.1. Expected Error Patterns

Before inserting the final examples, the main hypotheses are:

1. *Ambivalent* remains the hardest class.
2. *Ambivalent* and *Clear Non-Reply* are confused when an answer is topically related but not directly responsive.
3. long answers may mix responsive and evasive content, which makes a single global label harder.

Case	Gold Label	Predicted Label	Short Explanation / Example Snippet
1	[fill: label]	[fill: label]	[fill: insert one representative mistake from notebook]
2	[fill: label]	[fill: label]	[fill: insert one representative mistake from notebook]
3	[fill: label]	[fill: label]	[fill: insert one representative mistake from notebook]

Table 9: Representative errors to be copied from the notebook’s qualitative analysis cells.

8.2. Representative Mistakes

A good final write-up should mention not only what was wrong, but also why the example is difficult. In particular, the best examples are those where the answer contains lexical overlap with the question but still fails to directly address it.

9. Discussion

This Homework 2 design deliberately avoids heavy optimization. That is a feature rather than a weakness. The point of the assignment is to demonstrate understanding of contextual modeling, not to maximize a leaderboard score through large-scale tuning.

The key methodological differences from Homework 1 are:

- raw question–answer pairs instead of hand-normalized text,
- sentence-pair tokenization instead of concatenated sparse features,
- contextual sequence classifiers instead of Logistic Regression,
- one fixed validation split instead of 3-fold cross-validation.

This creates a clean scientific comparison. If Homework 2 improves on Homework 1, the report can reasonably attribute the gain to contextual sentence-pair modeling rather than to a much larger tuning effort.

9.1. Small Non-Improving Changes

This subsection can be completed with one or two compact observations from the sweep. Example fill-in patterns:

- Increasing max length from *[fill: value]* to *[fill: value] [fill: did / did not]* help, which suggests *[fill: brief interpretation]*.
- Training for 4 epochs instead of 3 *[fill: helped / did not help]*, which suggests *[fill: underfitting / overfitting / similar convergence]*.

10. Conclusion

Homework 2 extends Homework 1 in the most direct possible way: it preserves the high-level pipeline but replaces the non-contextual source representation with a contextual transformer encoder and replaces the linear classifier with an end-to-end fine-tuned sequence classifier. This keeps the comparison between the two homeworks clean and easy to explain.

After the final GPU run, the conclusion should answer three questions in compact form:

1. Did contextual fine-tuning outperform the Homework 1 baselines?
2. Which of the three transformer checkpoints was strongest?
3. What remained difficult about the task even for contextual models?

A ready-to-fill concluding sentence is:

Overall, Homework 2 shows that *[fill: best model] [fill: did / did not]* provide a clear improvement over the Homework 1 baselines. The main remaining challenge is *[fill: hardest confusion or subgroup]*, which indicates that response-clarity classification still requires nuanced reasoning about relevance and evasiveness beyond surface lexical overlap.

References

- [1] AILS-NTUA. Question evasion detection in political interviews. HuggingFace dataset: <https://huggingface.co/datasets/ailsntua/QEvasion>, 2024.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.
- [3] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.
- [4] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

- [5] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.